

Instructions how to set up website

Step 1: Set Up the Project Environment

Step 1.1: Install Required Software

- **Node.js:** Download and install Node.js from <https://nodejs.org/>.
- **Git:** Download and install Git from <https://git-scm.com/>.
- **Heroku CLI:** Download and install the Heroku CLI from <https://devcenter.heroku.com/articles/heroku-cli>.
- **MongoDB Atlas:** Sign up for a free account at <https://www.mongodb.com/cloud/atlas>.

Step 1.2: Set Up a New Node.js Project

1. Initialize Node.js project:
 - Open a terminal or command prompt.
 - Navigate to your project directory.
 - Run `npm init -y` to create a package.json file.
2. Install required dependencies:
 - Run the following command to install the necessary packages:
`npm install express body-parser mongoose cors path socket.io`

Step 1.3: Set Up the Project Structure

Create the following files and directories in your project directory:

- **index.html:** The main HTML file for the front end.
- **stylesheets/style.css:** The CSS file for styling the website.
- **js/script.js:** The JavaScript file for handling client-side logic.
- **js/map.js:** The JavaScript file for handling the map logic (if applicable).
- **server.js:** The main Node.js server file.
- **Procfile:** A file used by Heroku to start your application.

Step 1.4: Set Up MongoDB Atlas

1. Create a Cluster:
 - Log in to your MongoDB Atlas account.
 - Create a new cluster.
2. Create a Database User:
 - Create a new database user with a password.
 - Make sure to keep the username and password for later use.
3. Obtain the Connection String:
 - From your cluster, click on "Connect" and select "Connect your application".
 - Copy the connection string.

Step 1.5: Set Up Heroku

1. Log in to Heroku:
 - Open your Node js terminal and log in to Heroku using the following command:

`heroku login`
 - This will open a browser window where you can log in to your Heroku account.
2. Create a New Heroku App:
 - In the terminal, navigate to your project directory.
 - Create a new Heroku app by running:

`heroku create your-app-name`

Step 2: Develop the Project

Step 2.1: Create the Front-End (HTML, CSS, JavaScript)

- **index.html:** Create the structure of your webpage.
- **stylesheets/style.css:** Style your webpage using CSS.
- **js/script.js:** Handle data fetching, updating the DOM, and interacting with the server.
- **js/map.js:** (Optional) Handle the map functionality.

Step 2.2: Create the Back-End (Node.js)

- **server.js:**
 - Import necessary libraries.
 - Set up Express.js to serve the HTML file and handle API requests.
 - Connect to MongoDB using Mongoose.
 - Create a schema and model for your data.
 - Set up API routes to save and retrieve data.
 - Set up socket.io for real-time updates.
 - Start the server.

Step 2.3: Add the Heroku Configuration

- **Create file Procfile in project folder:** Add the following line to specify the command to start your app: `web: node server.js`

Step 3: Deploy to Heroku

Step 3.1: Initialize a Git Repository

- If you haven't already, initialize a Git repository in your project directory, run those commands in order:

`git init`

`git add .`

`git commit -m "Initial commit"`

Step 3.2: Deploy to Heroku

- Add the Heroku remote to your Git repository:

`heroku git:remote -a your-app-name`

- Deploy your app to Heroku by running:

`git push heroku master`

Step 3.3: Configure MongoDB Environment Variable

- Set the MongoDB connection string as an environment variable in Heroku:

`heroku config:set MONGODB_URI="your-mongodb-connection-string"`

Step 3.4: Open the Deployed App

- After deployment, you can open your app in the browser:

[heroku open](#)

Step 4: Testing and Validation

Step 4.1: Test the Web Interface

- Open the web application and check if the data is displayed correctly.
- Test all functionalities, including real-time data updates, data history, and API data display.

Step 4.2: Test the Node.js Server

- Test the server endpoints using tools like Postman to ensure that data is correctly saved and retrieved.

Step 4.3: Test MongoDB Integration

- Ensure that the data is correctly stored in your MongoDB Atlas database by checking the collections.